Rapport de soutenance 1 - Oxide

Alexandre Joaquim Lima Salgueiro
alexandre-joaquim.lima-salgueiro

Romain Bailly
romain.bailly

Willian Huang Hong
william.huang-hong

Tom Huynh
tom.huynh

13 mars 2026

Sommaire

1 Rôle et mission de chaque membre **1**
1.1 Tom Huynh . . . 1
1.1.1 Mission principale . . . 1

2 Objectifs et état d'avancement **2**
2.1 Tom Huynh . . . 2
2.1.1 Objectifs du projet . . . 2

3 Choix technologiques et algorithmiques **3**
3.1 Tom Huynh . . . 3
3.1.1 Conception du format natif `.oxz` . . . 3
3.1.2 Intégration de la compression universelle . . . 4
3.1.3 Analyse et aiguillage dynamique des données . . . 4
3.1.4 Architecture de traitement parallèle et optimisations système . . . 5

1 Rôle et mission de chaque membre

1.1 Tom Huynh

1.1.1 Mission principale

En tant que responsable de l'infrastructure technique, ma mission consiste à définir et à mettre en œuvre le socle logiciel du projet Oxide. Mon rôle s'articule autour de la conception du format de fichier et de l'orchestration des flux de données, garantissant ainsi la performance et la fiabilité de la chaîne de traitement.

Responsabilités et axes de développement

- Conception et implémentation du format natif `.oxz` (manifeste, métadonnées, blocs de données) ;
- Intégration et orchestration des algorithmes de compression dans la chaîne de traitement ;
- Mise en place du système de routage intelligent basé sur le type de contenu ;
- Développement d'une architecture de traitement parallèle pour l'extraction et la compression.

2 Objectifs et état d'avancement

2.1 Tom Huynh

2.1.1 Objectifs du projet

Pour la première soutenance, les objectifs portent principalement sur l'infrastructure du projet. Ils concernent la mise en place du format natif `.oxz`, du routage des données dans le pipeline de traitement, ainsi que l'intégration des premiers algorithmes de compression.

Éléments attendus et état pour la première soutenance

- Un format `.oxz` fonctionnel, utilisable pour l'archivage et l'extraction ;
- Une logique de routage capable d'identifier le type global d'un fichier et de préparer son pré-traitement ;
- Un pipeline parallèle de compression et d'extraction en place ;
- L'intégration de la compression universelle avec `LZ4` ;

3 Choix technologiques et algorithmiques

3.1 Tom Huynh

3.1.1 Conception du format natif `.oxz`

Le développement du format `.oxz` a été dicté par la nécessité de disposer d'une structure de fichier flexible, capable de stocker des métadonnées riches tout en permettant un accès rapide aux données. L'objectif était de s'écarter d'une lecture séquentielle stricte pour privilégier un accès indexé aux différents segments de l'archive.

Organisation et structure du conteneur

Le format repose sur une organisation linéaire structurée autour de zones clés : un en-tête global, un manifeste pour la structure logique, une table des blocs pour la gestion physique, et enfin le stockage des données utiles (*payloads*).

L'en-tête principal contient directement l'emplacement des sections importantes. Cela permet de retrouver rapidement les informations utiles sans devoir relire tout le fichier dans l'ordre.

Le manifeste sert à décrire la structure logique de l'archive. C'est lui qui permet de savoir quels fichiers et quels répertoires doivent être recréés à l'extraction.

La table des blocs décrit ensuite la partie physique du stockage. Chaque entrée indique où se trouve un segment de données, quelle taille il occupe, et quels traitements lui ont été appliqués.

Les données sont ensuite stockées à la suite, dans un ordre stable et contigu. Enfin, le pied de page (*footer*) ferme l'archive avec une signature de fin et une valeur de contrôle globale.

```
EN-TÊTE GLOBAL (48 octets)
   signature = "OXZ\0"
   version = 1
   réservé (2 octets)
   positions (métadonnées, manifeste, table blocs, données, footer)

MÉTADONNÉES DE L'ARCHIVE (8 octets)
   signature = "OXMD"
   version = 1
   source = fichier | dossier

MANIFESTE DES ENTRÉES
   signature = "OXMF"
   version = 1
   nombre d'entrées
   entrées[]
      type = dossier | fichier
      chemin (longueur + contenu UTF-8)
      taille, mode, mtime
      position du contenu utile

TABLE DES BLOCS
   signature = "OXCI"
   version = 1
   nombre de blocs
   descripteurs de blocs[]
      offset relatif du payload
      tailles (compressée et originale)
      somme de contrôle (CRC32)
      indicateurs (compression et stratégie)

SECTION DES DONNÉES
   bloc_0, bloc_1, ..., bloc_n-1

PIED DE PAGE (8 octets)
   signature = "END\0"
   checksum globale
```

3.1.2 Intégration de la compression universelle

Un algorithme de compression est dit "universel" lorsqu'il est capable de réduire la taille de n'importe quel flux de données sans connaissance préalable de ses propriétés statistiques. Contrairement aux filtres spécialisés qui exploitent la sémantique de types de fichiers spécifiques (images, texte, audio), il s'appuie sur des modèles mathématiques généraux pour identifier et éliminer les redondances, garantissant une efficacité robuste quel que soit le contenu traité.

L'implémentation initiale repose sur l'algorithme `LZ4`. Ce choix permet de valider l'architecture du pipeline avec une solution robuste, tout en garantissant des performances optimales.

Détails de fonctionnement et implémentation

`LZ4` appartient à la famille des algorithmes LZ77 et se distingue par sa conception orientée vers la performance brute. Son fonctionnement repose sur une structure de "blocs" composée de séquences de littéraux et de copies de données déjà rencontrées :

- Chaque séquence commence par un octet de contrôle (token). Les 4 bits de poids fort indiquent la longueur des littéraux qui suivent. Si cette longueur dépasse 15, des octets supplémentaires sont ajoutés. Les littéraux sont ensuite copiés tels quels du flux d'entrée vers la sortie.
- Les 4 bits de poids faible du token indiquent la longueur de la répétition. Une répétition est définie par un décalage arrière de 16 bits, pointant vers une séquence identique déjà traitée dans un dictionnaire glissant de 64 KiB.
- Contrairement à des algorithmes comme Deflate, LZ4 n'utilise aucun codage entropique final (comme Huffman). Le traitement s'effectue quasi exclusivement par des copies de mémoire et des manipulations d'octets entiers, évitant les opérations coûteuses au niveau du bit.

Cette architecture permet à LZ4 d'atteindre des débits de compression et de décompression symétriques extrêmement élevés, limités principalement par la bande passante de la mémoire vive, comme illustré dans les mesures de performance suivantes :

Jeu de données	**Compression**	**Décompression**
Texte 1 MiB	13.4 GiB/s	9.9 GiB/s
Mixte 1 MiB	13.0 GiB/s	14.1 GiB/s
Aléatoire 1 MiB	2.9 GiB/s	13.1 GiB/s

3.1.3 Analyse et aiguillage dynamique des données

Le système de routage analyse les flux entrants pour déterminer dynamiquement la chaîne de traitement la plus adaptée. Cette approche sensible au contenu permet d'appliquer des pré-traitements spécialisés (images, texte, binaire) avant la compression finale, optimisant ainsi le ratio global de l'archive.

Stratégie de détection multi-niveaux

Le routage repose sur un système de détection hybride (*FormatDetector*) qui combine la précision des signatures binaires avec la flexibilité d'analyses heuristiques :

- **Signatures binaires :** Utilisation de la bibliothèque *infer* pour identifier les formats standards (JPEG, PNG, WAV, etc.) via leurs *magic numbers* ;
- **Analyse heuristique du texte :** Évaluation de la validité UTF-8 et calcul des ratios de caractères imprimables vs octets de contrôle sur un échantillon de 16 KiB ;
- **Heuristique binaire :** Recherche de prologues de fonctions x86/x86_64 (ex: `push rbp; mov rbp, rsp`) pour identifier les exécutables non signés ;
- **Performance :** L'analyse est strictement limitée aux premiers 64 KiB de chaque fichier, garantissant une détection quasi-instantanée quelle que soit la taille du fichier.

Logique de segmentation et d'aiguillage

L'identification de la catégorie de fichier définit une stratégie de découpage alignée sur la structure des données :

- **Texte :** Recherche du caractère de saut de ligne (`\n`) le plus proche de la taille cible pour préserver l'intégrité des lignes ;
- **Image :** Calcul de la taille d'une ligne de pixels à partir des métadonnées (largeur $\times$ octets par pixel). Les segments sont systématiquement alignés sur des multiples de cette taille pour garantir que chaque bloc contient des lignes complètes ;
- **Audio :** Identification de la taille des trames PCM (canaux $\times$ profondeur de bit $\times$ échantillons par paquet). Le découpage respecte ces frontières pour éviter de couper une trame sonore ;
- **Binaire/Générique :** Découpage brut à la taille cible sans alignement spécifique.

Cette organisation permet aux algorithmes de pré-traitement de travailler sur des unités logiques cohérentes, maximisant l'efficacité des transformations (ex: YCoCg-R sur des lignes d'images entières) plutôt que sur des suites d'octets arbitraires.

3.1.4 Architecture de traitement parallèle et optimisations système

L'infrastructure d'Oxide est conçue pour une saturation maximale des ressources matérielles à travers plusieurs couches d'optimisation, allant de la gestion des fils d'exécution jusqu'au contrôle fin des accès disque.

Algorithme de Work-Stealing

Le pipeline repose sur un modèle de parallélisme par tâches orchestré par un pool de threads. L'algorithme de *work-stealing* assure un équilibrage dynamique de la charge via une architecture à double file d'attente :

- **Gestion des files :** Chaque thread possède une file locale. Cela favorise la localité du cache en traitant les données les plus récentes. Une file globale reçoit les nouvelles tâches du thread principal ;
- **Stratégie de vol :** Un thread inactif tente d'abord de vider sa file locale, puis la file globale. S'il reste inoccupé, il devient un « voleur » et récupère une tâche au fond de la file d'un autre thread. Cette approche réduit la contention entre le propriétaire de la file et les voleurs tout en garantissant qu'aucun cœur ne reste inactif tant qu'il reste du travail.

Réordonnancement séquentiel et déterminisme

Malgré un traitement concurrent où les blocs peuvent être finalisés dans un ordre arbitraire (ex: le bloc 10 finit avant le 1), ainsi on garantit un format de sortie strictement déterministe grâce à un algorithme de réordonnancement par fenêtre glissante :

- **Tampon de réordonnancement :** Un gestionnaire stocke les blocs terminés dans une structure de données triée (`BTreeMap`). Si un bloc arrive hors-séquence, il est mis en attente jusqu'à ce que tous ses prédécesseurs soient complétés ;
- **Libération contiguë :** Dès que le bloc attendu arrive, le système libère immédiatement ce bloc ainsi que tous les blocs suivants déjà présents en mémoire, permettant une écriture disque **strictement séquentielle**. Ce mécanisme inclut un contrôle de flux pour éviter que les threads de calcul ne saturent la mémoire vive en attendant le thread d'écriture.

Optimisations des entrées/sorties (I/O)

La performance est souvent limitée par le stockage ; Ainsi on intègre plusieurs stratégies pour lever ces verrous :

- **Memory Mapping (mmap) :** L'accès aux fichiers sources utilise les fonctions de projection mémoire du kernel. Cela permet des lectures à très haute vitesse sans copie intermédiaire entre l'espace noyau et l'espace utilisateur (*zero-copy*) ;
- **Buffer Pooling :** Pour éviter les ralentissements liés aux allocations fréquentes, Oxide réutilise systématiquement des zones mémoire pré-allouées via un pool de tampons, réduisant la pression sur l'allocateur système.